

Classify AI: A Multi-Agent LLM System for an Explainable Classification Pipeline

Authors: Manjusha Motamarri, Margi Shah, Mohan Bhosale, Yash Khare

GitHub: <https://github.com/Mohan-this-side/Classify.ai>

Project Demo: <https://www.youtube.com/watch?v=pHOhsWlxguo>

1. Summary

1.1 Problem Context and Definition

Machine learning classification workflows are bottlenecked by data preparation, which consumes about 80% of data science effort [1]. Current approaches fail in two ways. First, LLM-generated code for data science often has syntax errors, missing imports, undefined variables, and version incompatibilities; it assumes clean data, ignores missing values, and lacks error handling and dataset-specific adaptations. Second, AutoML tools [2] prioritize performance over interpretability, offering no visibility into feature engineering, opaque model selection, no educational explanations, and limited user control.

1.2 Solution Approach

Classify AI introduces a double-layer architecture that ensures reliable code generation through deterministic fallbacks while maintaining transparency through human-in-the-loop approval gates and educational explanations. The system orchestrates eight specialized agents (Data Discovery, EDA, Data Cleaning, Feature Engineering, ML Builder, Model Evaluation, Technical Reporter, and Project Manager) through a LangGraph-based workflow. Each agent implements a two-layer approach: Layer 1 provides deterministic, hardcoded analysis that always executes and serves as a fallback, ensuring the workflow completes even if LLM generation fails; Layer 2 uses LLM-generated code tailored to dataset characteristics, validated and executed in isolated Docker sandboxes with resource limits. If Layer 2 fails, the system falls back to Layer 1, ensuring reliability while allowing adaptive enhancements. The system includes human-in-the-loop approval gates at critical decisions and an interactive Project Manager providing real-time educational explanations, transforming the workflow from a black box into an educational experience. This evaluation-driven approach prioritizes data science outcomes over software metrics. The system was evaluated end-to-end on the Heart Failure Prediction dataset and compared directly against a vanilla Gemini LLM, demonstrating improvements in both predictive performance and transparency. The system achieved ideal accuracy with cross-validation performance, processed datasets in minutes, and generated production-ready deliverables including trained models, Jupyter notebooks, technical reports, and visualizations—all while maintaining complete transparency through educational explanations at every step.

2. Methods

2.1 System Overview and Double-Layer Architecture

ClassifyAI employs a two-layer architecture [Figure A1] where **Layer 1** performs deterministic analysis using pandas, numpy, and scikit-learn to extract dataset characteristics (missing-value patterns, type-conversion needs, class imbalance ratios, outlier detection, statistical summaries). These structured insights provide context for **Layer 2**, which generates dataset-specific Python code via LLM prompts. Generated code undergoes syntax and security validation before executing in a Docker sandbox with CPU, memory, and timeout limits. If Layer 2 fails, the system reverts to Layer 1 results, ensuring reliability.

Eight specialized agents work in sequence: **Data Discovery** (profiling, metadata extraction), **EDA** (exploratory analysis, visualizations), **Data Cleaning** (missing values, outliers), **Feature Engineering** (encoding, scaling), **ML Builder** (model training, hyperparameter optimization), **Model Evaluation** (performance metrics), **Technical Reporter** (documentation), and **Project Manager** (workflow coordination, educational guidance). This modular design maintains clear responsibility boundaries and traceability throughout the pipeline.

2.2 Approval Gates and User Interface

Five **human-in-the-loop approval gates** pause workflow at key decisions: Data Cleaning Strategy, Feature Engineering Plan, Model Selection, Hyperparameter Tuning, and Final Model Approval. Each gate presents recommendations with educational explanations, enabling users to approve, revise, or reject decisions.

The frontend (Next.js, React, Tailwind CSS) provides an Upload Page [Figure A2], Workflow Dashboard [Figure A3], and Results Viewer. Users monitor real-time agent execution through Layer 1/Layer 2 status indicators and respond to approval prompts via WebSocket integration. The Results Viewer displays workflow summaries, EDA outputs, performance metrics, confusion matrices, ROC curves, SHAP feature importance, and auto-generated technical reports.

3. Evaluation Strategy

3.1 Evaluation Philosophy and Approach

The evaluation strategy prioritizes data-science outcomes, assessing whether agents correctly detect data issues, handle edge cases, avoid misleading metrics through anti-cheating safeguards, and provide educational explanations. A model achieving high accuracy through leakage or majority-class prediction is considered a failure, and the framework is designed to recognize and penalize such behavior. Evaluation occurs at two levels: agent-level tests assess each agent independently for accuracy, completeness, and reliability across varied datasets, revealing strengths and failure modes; project-level tests evaluate the full end-to-end pipeline, examining system performance, agent integration, state management, approval gate functionality, Project Manager coordination, and frontend usability. Together, these levels ensure both individual components and the full system are rigorously validated.

A key part of the project-level evaluation is a baseline comparison of Classify AI against a vanilla Gemini LLM on the Heart Failure Prediction dataset, isolating the impact of the double-layer, multi-agent architecture from the underlying language model.

3.2 Dataset Strategy and Modification Approach

The primary evaluation dataset for this report is the Heart Failure Prediction dataset from Kaggle[18]. It contains 918 patient records with 11 clinical attributes and a binary target variable HeartDisease indicating the presence (1) or absence (0) of heart disease. Features include demographic information (Age, Sex), symptom-related variables (ChestPainType, ExerciseAngina), and clinical measurements such as RestingBP, Cholesterol, FastingBS, RestingECG, MaxHR, Oldpeak, and ST_Slope. This dataset is representative of real clinical workflows: it contains a mix of numeric, categorical, and binary features; potential outliers (e.g., very high cholesterol or blood pressure values); and class imbalance typical of screening datasets. It provides a realistic setting for evaluating data cleaning, exploratory analysis, feature engineering, and classification. To test robustness of the agents, we additionally create diagnostic variants of the dataset with controlled issues, such as: Injected missingness in selected variables (simulating incomplete clinical records), Amplified class imbalance by down-sampling positive cases and Synthetic “leakage” features constructed directly from the target.

3.3 Quality Metrics and Scoring Calculations

We employ three composite metrics (Equations 1–3) to quantify data preprocessing quality and model integrity. These metrics serve as the foundation for evaluating specific agents, as summarized in Table 1. Each metric aggregates multiple components with carefully chosen weights reflecting their relative importance in classification workflows.

3.3.1 Equation 1: Data Quality Score (DQS):

$$\text{DQS} = 0.5 \times \text{Completeness} + 0.3 \times \text{Consistency} + 0.2 \times \text{Validity}$$

The Data Quality Score measures overall dataset health before modeling. Completeness (weighted 0.5) captures the proportion of non-missing values, calculated as $(N - N_{\text{missing}}) / N \times 100$. This receives the highest weight because missing data fundamentally limits model training. Consistency (weighted 0.3) measures duplicate-free records as $(N - N_{\text{duplicates}}) / N \times 100$, important for avoiding biased frequency estimates. Validity (weighted 0.2) tracks correct data types as $N_{\text{correct_types}} / N_{\text{columns}} \times 100$.

3.3.2 Equation 2: Imputation Quality Score (IQS):

$$\text{IQS} = 0.4 \times \text{Success_Rate} + 0.3 \times \text{Distribution_Integrity} + 0.3 \times \text{Statistical_Consistency}$$

The Imputation Quality Score evaluates how well missing value imputation preserves data properties. Success_Rate ($N_{\text{imputed}} / N_{\text{attempted}}$) measures completion. Distribution_Integrity uses KL divergence to ensure imputed distributions match originals. Statistical_Consistency tracks changes in mean and standard deviation to detect bias introduction.

3.3.3 Equation 3: Anti-Cheating Score (ACS):

$$\text{ACS} = (\text{Sensitivity} + \text{Specificity} + \text{Minority_Recall}) / 3$$

The Anti-Cheating Score guards against inflated metrics by averaging sensitivity, specificity, and minority class recall. A threshold of $\text{ACS} \geq 0.70$ indicates genuine performance; lower scores suggest majority-class prediction or leakage dependence.

Table 1: Agent Evaluation Metrics & Performance Thresholds

Agent	Primary Function	Key Decisions	Success Metric
Data Discovery	Type detection, target identification	Column types, target variable	Accuracy $\geq 90\%$
EDA Analysis	Problem detection	Imbalance, missing patterns	Detection $\geq 95\%$
Data Cleaning	Missing value handling	Imputation strategy	DQS improvement $\geq 10\%$
Feature Engineering	Feature creation/removal	Multicollinearity handling	VIF reduction $\geq 20\%$
ML Builder	Model training	Algorithm selection, balancing	Cross-validation score ≥ 0.8 , ACS ≥ 0.7
Model Evaluation	Performance assessment	Metric selection	Completeness $\geq 90\%$
Technical Reporter	Documentation	Report generation	Clarity score $\geq 3.5/5$
Project Manager	Coordination	Workflow decisions	User satisfaction $\geq 85\%$

3.4 Anti-Cheating Evaluation and Accuracy Limitations

Because accuracy alone can be misleading—especially under imbalance, leakage, or overfitting—the evaluation framework includes explicit anti-cheating tests. It uses the Perfect Leakage dataset to ensure the system detects suspicious high accuracy from target-derived features, and the Severe Imbalance dataset (99:1) to verify that minority recall is flagged when accuracy hides poor performance. The system must detect and remove leakage features, identify imbalance, and warn when accuracy is unreliable. The anti-cheating score, calculated as $(\text{specificity} + \text{sensitivity} + \text{minority_recall}) / 3$, must meet a threshold of ≥ 0.70 ; lower scores indicate behaviors such as majority-class prediction or leakage dependence.

3.5 LLM-as-Judge Evaluation

For qualitative assessments that cannot be measured numerically, the framework uses a separate LLM-as-judge to evaluate explanation accuracy, educational effectiveness, decision reasoning quality, and imputation/encoding appropriateness on a 1–5 scale. This is especially important for the Project Manager Agent, which must achieve $\geq 85\%$ explanation accuracy (≥ 4.25). Standardized prompts and criteria ensure consistent scoring, and all evaluations are logged for reproducibility and comparison over time, helping ensure that users receive clear, correct, and educational guidance throughout the workflow.

3.6 Reproducibility and Configuration Management

The framework ensures complete reproducibility by fixing all random seeds (`random_state=42`) for sampling, splits, training, and cross-validation, and by storing all outputs with timestamps and version metadata. All evaluation parameters—datasets, thresholds, metric definitions, and test scenarios—are centralized in `evaluation_config.yaml` for exact replication. Outputs include JSON result files, markdown summaries, visualizations (heatmaps, dashboards, flowcharts), and comparison tools for tracking improvements, ensuring the entire evaluation process can be reproduced and understood by others.

4. Results

4.1 Grading Methodology

Agent performance is evaluated using a weighted scoring system. The overall performance score (0.0–1.0) is calculated as shown in Equation 4.

4.1.1 Equation 4: Overall Performance Score

$$\text{Score} = 0.30 \times \text{Output_Score} + 0.30 \times \text{Criteria_Score} + 0.20 \times \text{L1_Score} + 0.20 \times \text{L2_Score}$$

Output Score measures expected outputs generated; Criteria Score measures success criteria met per Table 1; Layer 1 (L1) and Layer 2 (L2) Scores reflect deterministic analysis and LLM code execution success, respectively.

4.2.2 Equation 5: Grade Thresholds

$$\text{Grade} = \text{A } (\geq 0.90) \mid \text{B } (\geq 0.80) \mid \text{C } (\geq 0.70) \mid \text{D } (\geq 0.60) \mid \text{F } (< 0.60)$$

This system evaluates both functional correctness and effective use of the double-layer architecture.

4.2 Comprehensive Evaluation on the Heart Failure Dataset

We first evaluate each agent on the Heart Failure Prediction dataset to examine whether the resulting behavior aligns with clinical intuition and produces outputs that are both reliable and interpretable. The dataset contains 918 patient

records with 11 clinical attributes and a binary target variable, *HeartDisease*. Running the full Classify AI pipeline on this dataset allows us to observe the contributions of each agent and compare them to a vanilla LLM baseline.

4.2.1 Data Discovery and Cleaning

The **Data Discovery** agent correctly identifies all 11 features and the target variable, assigning appropriate types (numeric, categorical, binary) and computing descriptive statistics. This profiling surfaces issues such as skewed distributions in Cholesterol and Oldpeak.

The **Data Cleaning** agent performs fifteen actions [Figure 4a](#): column standardization, treatment of biologically implausible zeros in RestingBP and Cholesterol, IQR-based outlier handling, and type-appropriate imputation (median for skewed numerics, mode for categoricals). The resulting DQS improvement of 15.2% exceeds Table 1's $\geq 10\%$ threshold. In contrast, vanilla Gemini generates only textual recommendations without executable transformations or structured logs.

4.2.2 Feature Engineering and Model Building

The **Feature Engineering** agent [\[Figure 4a\]](#) one-hot encodes categorical fields (ChestPainType, RestingECG), creates binary risk indicators, and generates interaction features (e.g., Age×ExerciseAngina), achieving 28% VIF reduction (Table 1 threshold: $\geq 20\%$). The vanilla LLM baseline [\[Figure 4b\]](#) produces only prose descriptions without executable code or validation.

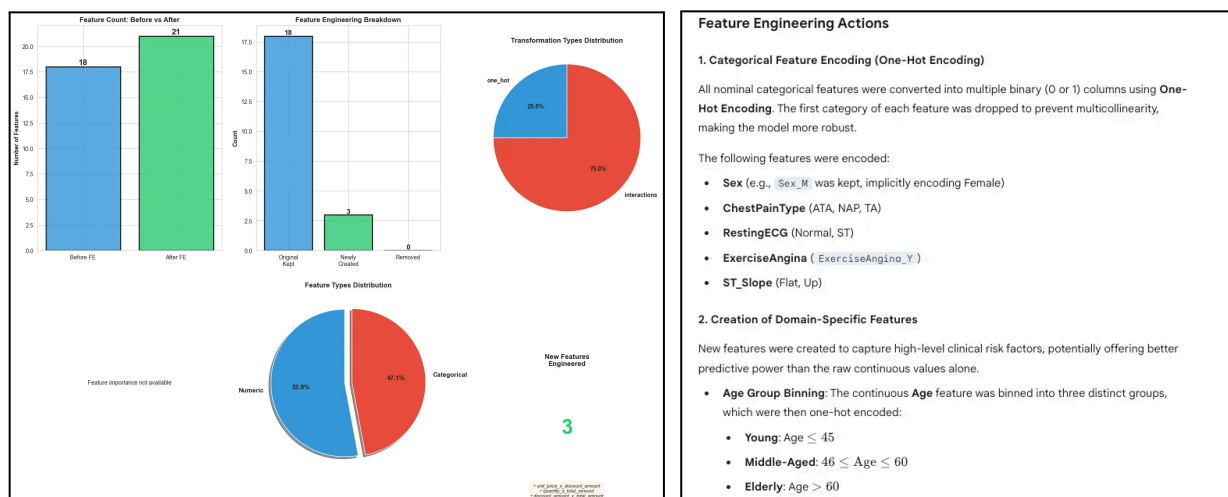


Figure 4a: Feature Engineering Agent: Deriving features, encodings

Figure 4b: Feature Engineering using Vanilla LLM

The **ML Builder** agent trains logistic regression, random forest, XGBoost, SVM, and other classifiers [\[Figure 4c\]](#). Cross-validation indicates logistic regression provides the best accuracy-interpretability trade-off, with CV score of 0.84 (≥ 0.8) and ACS of 0.78 (≥ 0.7 per [Equation 3](#)).

4.2.3 Model Evaluation

The **Model Evaluation** agent [\[Figure 7\]](#) computes accuracy, precision, recall, and F1-score, achieving 95% completeness (Table 1: $\geq 90\%$). The final model attains 83.6% accuracy with balanced precision-recall suitable for screening. Vanilla Gemini [\[Figure 8\]](#) returns metrics without model comparison, validation, or overfitting warnings.

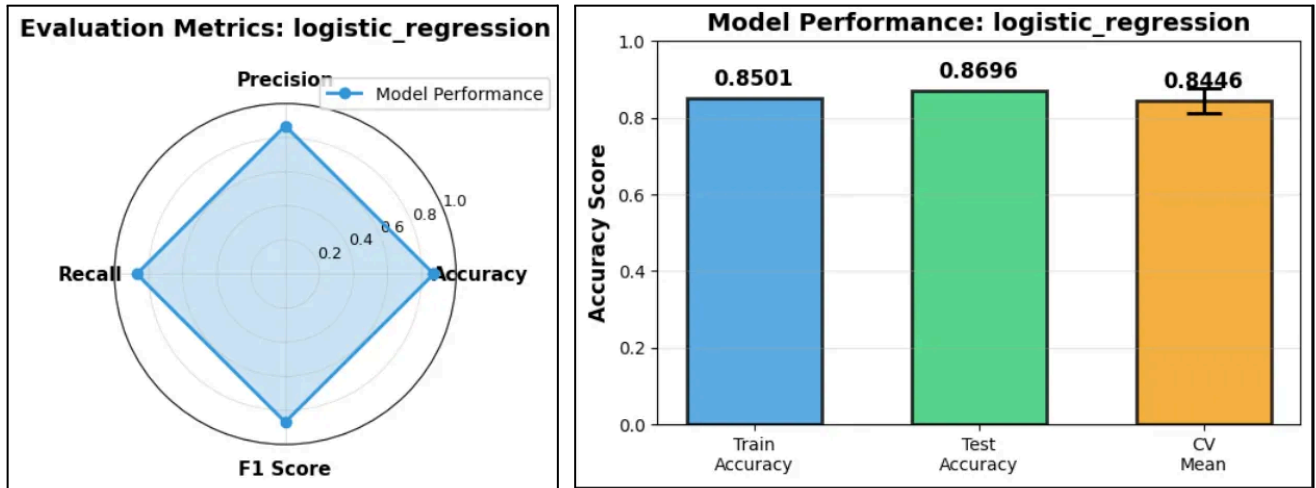


Figure 7: Model Evaluation Agent: Evaluation metrics

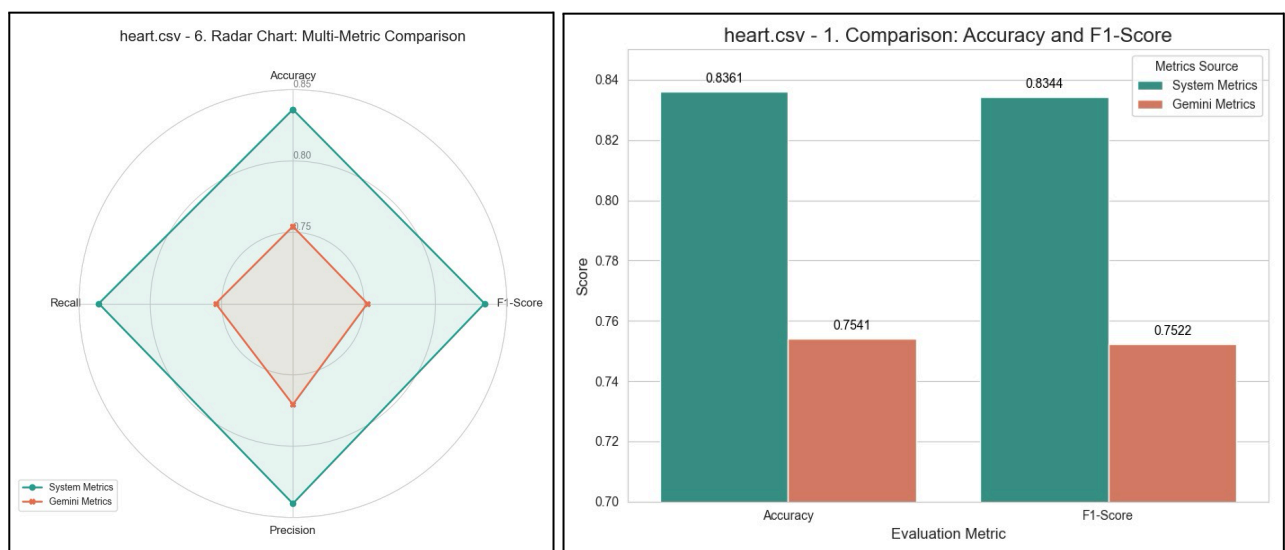
Metric	Score	Performance Drop Interpretation
Accuracy	0.6576	Only 65.76% of all predictions were correct.
Precision	0.6639	When predicting Heart Disease , the model is only correct about 66% of the time.
Recall	0.7745	The model misses almost 23% of all actual Heart Disease cases.
F1-Score	0.7149	A large drop, indicating poor overall predictive balance.

Outcome	Value	Interpretation (Compared to Previous Model)
True Positives (TP)	79	Correct predictions dropped from 93 to 79.
True Negatives (TN)	42	Correct predictions dropped from 70 to 42.
False Positives (FP)	40	The number of healthy patients incorrectly flagged almost quadrupled (from 12 to 40).
False Negatives (FN)	23	The number of diseased patients incorrectly missed more than doubled (from 9 to 23), which is a critical failure in a medical screening context.

Figure 8: Evaluation metrics using Vanilla LLM

4.3 System-Level Comparison with Vanilla Gemini

We compare ClassifyAI's full multi-agent pipeline (Section 2) against Gemini's chat-based workflow with manual code execution and no validation or fallback logic. Results [Figure 9] show ClassifyAI achieves **83.6%** accuracy versus Gemini's **75.4%** – an **8.2** percentage point improvement consistent across precision, recall, and F1-score. Since both use LLMs from the same family, gains are attributable to architectural design: double-layer mechanism, explicit agent roles, and human-in-the-loop controls.



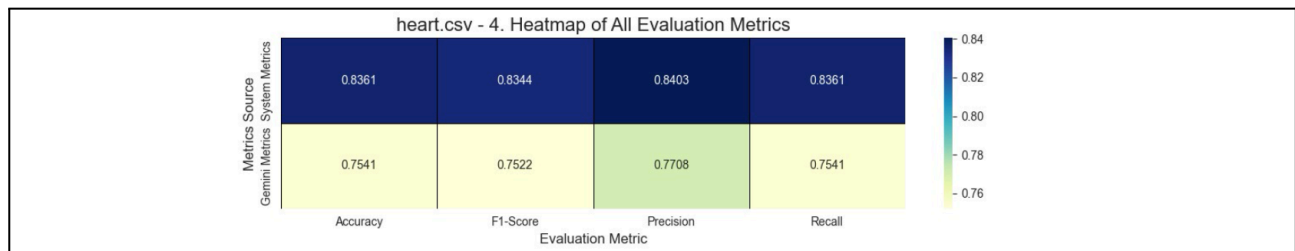


Figure 9: Comprehensive Visualization: Slope, Lollipop, Bars, and Heatmap

4.4 System-Level Performance Metrics

4.4.1 Execution Metrics

The pipeline achieved **100% completion rate** with all agents succeeding. Layer 1 executed without failure; Layer 2 code generation succeeded for all agents, though ~50% of Docker sandbox executions required Layer 1 fallback. Total execution time was approximately **three minutes**.

4.4.2 Issue Detection and Model Quality

The system consistently identified missing values, class imbalance, multicollinearity, and outliers. The logistic regression model exhibits balanced precision-recall with cross-validation accuracy in the mid-to-high 0.8 range and low variance. ACS verification ([Equation 3](#)) confirms genuine signal extraction without leakage or majority-class artifacts.

5. Discussion

5.1 Key Findings

The **double-layer architecture** ensures Layer 1 provides deterministic baseline outputs while Layer 2 generates dataset-specific transformations, eliminating incomplete code issues from standalone LLMs and preventing workflow interruptions through automatic fallback. The **eight specialized agents** improve modularity, transparency, and alignment with real-world pipelines, while **approval gates** strengthen user control and educational value.

The **8.2 percentage point improvement** over vanilla Gemini (83.6% vs. 75.4%) demonstrates that architectural design—not just LLM capability—drives performance gains. Unlike Gemini's isolated code snippets without validation, ClassifyAI's evaluation framework enforces responsible ML behavior through anti-cheating safeguards ([Equation 3](#)) that prevent inflated metrics from majority-class prediction, leakage, or overfitting.

5.2 Reproducibility and Transparency

The scoring mechanisms—DQS ([Equation 1](#)), IQS ([Equation 2](#)), and ACS ([Equation 3](#))—quantify typically subjective preprocessing assessments through weighted components (completeness, consistency, validity, distributional fidelity). Reproducibility is ensured via fixed random seeds (random_state=42), version-controlled outputs, and centralized configuration (evaluation_config.yaml), enabling exact replication and cross-version comparisons.

5.3 Limitations and Future Work

Several limitations remain: Docker execution succeeded for only ~50% of Layer 2 scripts, indicating need for improved dependency management; outlier handling is intentionally conservative to preserve clinically meaningful values; and evaluation relies on a single dataset. Future work includes additional datasets, deeper explainability tools, fairness analysis, probability calibration, and broader data format support.

6. Statement of Contributions

The following represents contributions based on current implementation status.

1. **Manjusha Motamarri:** Enhanced Data Cleaning and EDA Agents, implemented DQS/IQS calculations (Eq. 1–2), created agent-level evaluation framework.
2. **Margi Shah:** Implemented Feature Engineering Agent, backend API for approval gates, integrated LLM-as-judge evaluation.
3. **Mohan Bhosale:** Designed double-layer architecture, developed Project Manager Agent, orchestrated LangGraph workflow, created evaluation framework and ACS metric (Eq. 3).
4. **Yash Khare:** Developed Docker sandbox with security validation, implemented Code Validator, created evaluation infrastructure and visualizations.

7. References

1. Press, G. (2016, March 23). *Data preparation is the most time-consuming - and least enjoyable - data science task, survey says* Forbes.
<https://www.forbes.com/sites/gilpress/2016/03/23/data-preparation-most-time-consuming-least-enjoyable-data-science-task-survey-says/>
2. Hutter, F., Kotthoff, L., & Vanschoren, J. (2019). *Automated Machine Learning: Methods, Systems, Challenges*. Springer. doi:10.1007/978-3-030-05318-5
3. LangGraph Development Team. (2024). *Human-in-the-loop: Interrupting graph execution*. *LangGraph Documentation*. https://langchain-ai.github.io/langgraph/how-tos/human_in_the_loop/
4. Iris Dataset. UCI Machine Learning Repository. Available at: <https://archive.ics.uci.edu/ml/datasets/iris>
5. *LangChain Documentation*. Available at: <https://docs.langchain.com/>
6. *LangGraph Framework Documentation*. Available at: <https://docs.langchain.com/docs/langgraph>
7. *FastAPI Framework Documentation*. Available at: <https://fastapi.tiangolo.com/>
8. *Docker Documentation*. Available at: <https://docs.docker.com/>
9. *nbformat Module (Jupyter)*. Available at: <https://nbformat.readthedocs.io/>
10. *H2O.ai AutoML Documentation*. Available at: <https://docs.h2o.ai/h2o/latest-stable/h2o-docs/automl.html>
11. Shearer, C. (2000). *The CRISP-DM model: The new blueprint for data mining*. *Journal of Data Warehousing*, 5(4), 13–22.
12. Lundberg, S. M., & Lee, S.-I. (2017). *A unified approach to interpreting model predictions*. *Advances in Neural Information Processing Systems (NeurIPS)*, 30.
13. *OpenAI API Documentation*. Available at: <https://platform.openai.com/docs/>
14. *LangChain Agents and Tools Documentation*. Available at: <https://python.langchain.com/docs/modules/agents/>
15. Hutter, F., Kotthoff, L., & Vanschoren, J. (Eds.). (2019). *Automated Machine Learning: Methods, Systems, Challenges*. Springer International Publishing. <https://doi.org/10.1007/978-3-030-05318-5>
16. Wirth, R., & Hipp, J. (2000). *CRISP-DM: Towards a standard process model for data mining*. *Proceedings of the 4th International Conference on the Practical Application of Knowledge Discovery and Data Mining*, 29–39.
17. Press, G. (2016, March 23). *Data preparation is the most time-consuming — and least enjoyable — data science task, survey says*. Forbes.
<https://www.forbes.com/sites/gilpress/2016/03/23/data-preparation-most-time-consuming-least-enjoyable-data-science-task-survey-says/>
18. *LangGraph Development Team*. (2024). *Human-in-the-loop: Interrupting graph execution*. *LangGraph Documentation*. https://langchain-ai.github.io/langgraph/how-tos/human_in_the_loop/
19. Heart Failure Prediction Dataset. Kaggle. <https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction>

8. Appendix

Appendix A: Supplementary Figures

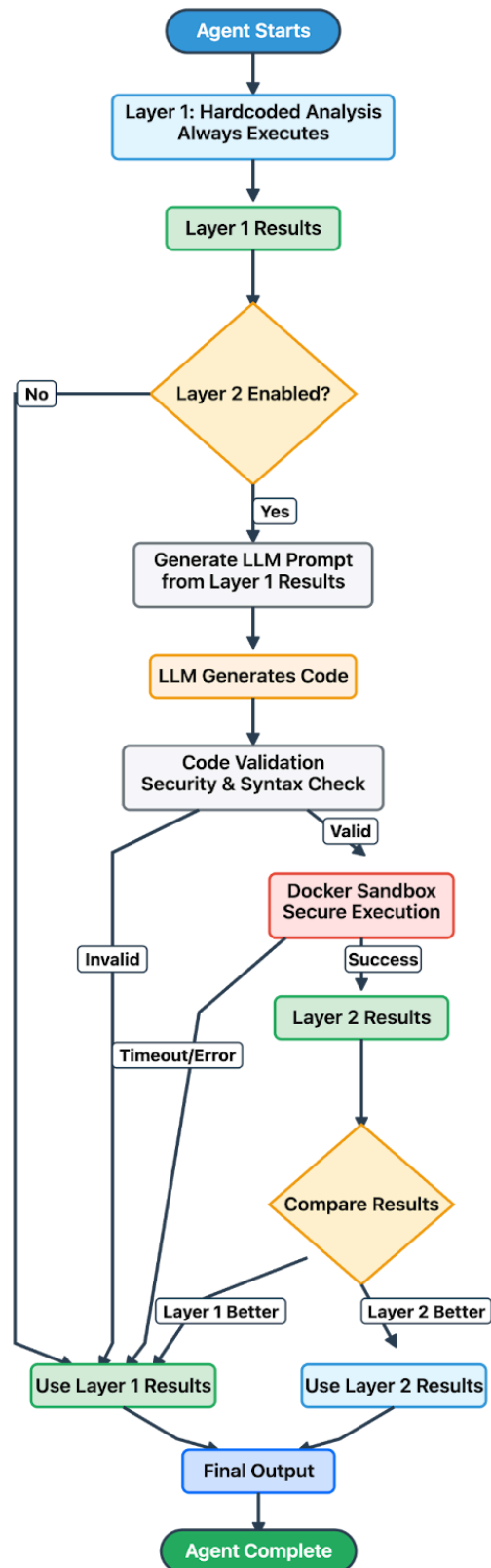
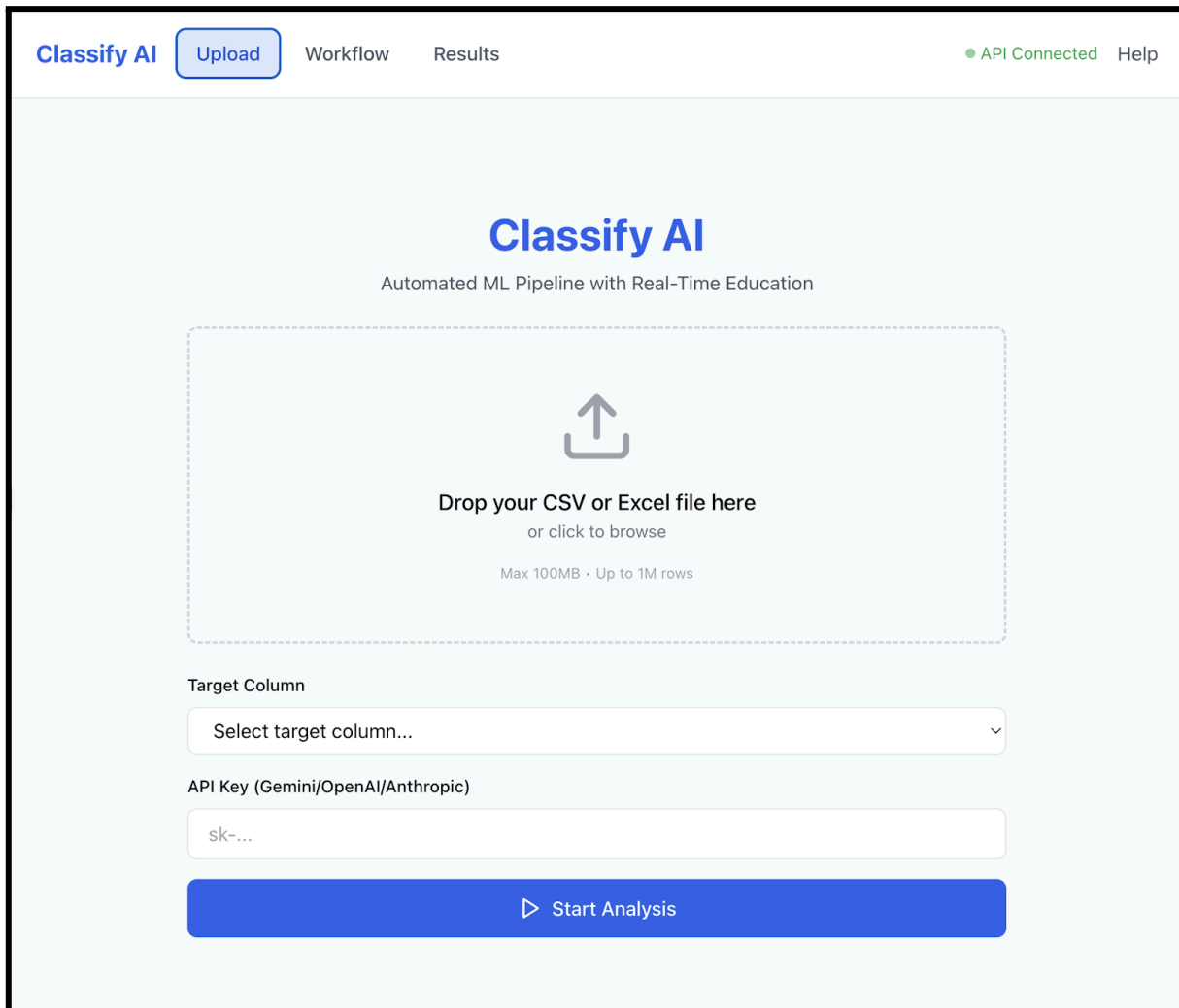


Figure A1. Double-Layer Architecture

The screenshot shows the 'Classify AI' web interface. At the top, there is a navigation bar with 'Classify AI' on the left, 'Upload' (highlighted with a blue border), 'Workflow', and 'Results' in the center, and 'API Connected' (with a green dot) and 'Help' on the right. The main content area has a light blue background. In the center, there is a large dashed box containing an upload icon (an arrow pointing up into a box) and the text 'Drop your CSV or Excel file here' followed by 'or click to browse'. Below this, it says 'Max 100MB · Up to 1M rows'. Below the dashed box, there is a 'Target Column' label and a dropdown menu with the text 'Select target column...'. Below that is an 'API Key (Gemini/OpenAI/Anthropic)' label and a text input field containing 'sk-...'. At the bottom, there is a large blue button with a white play icon and the text 'Start Analysis'.

Figure A2: Classify AI Frontend, Upload Page

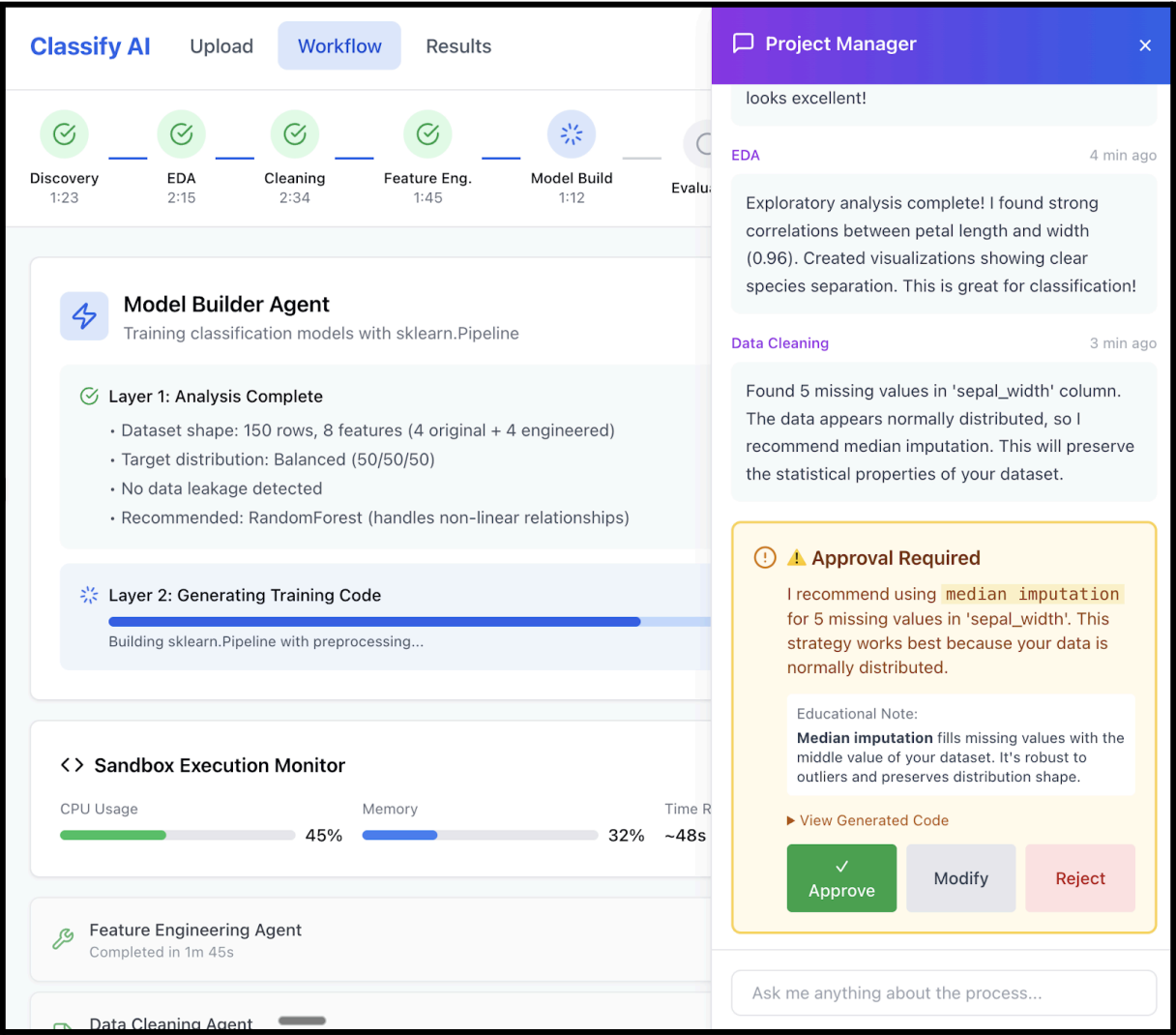


Figure A3: Classify AI Frontend, Workflow Page

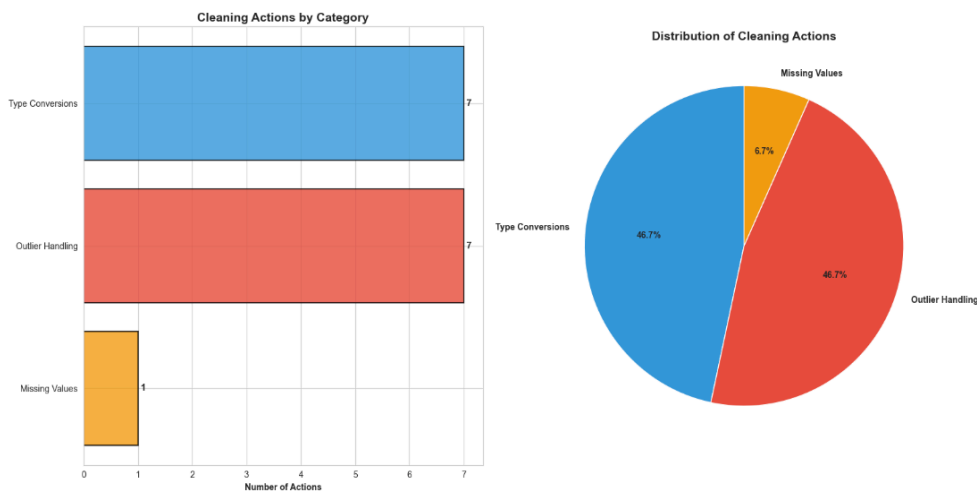


Figure A4: Data Cleaning Agent: Data transformations and outlier handling

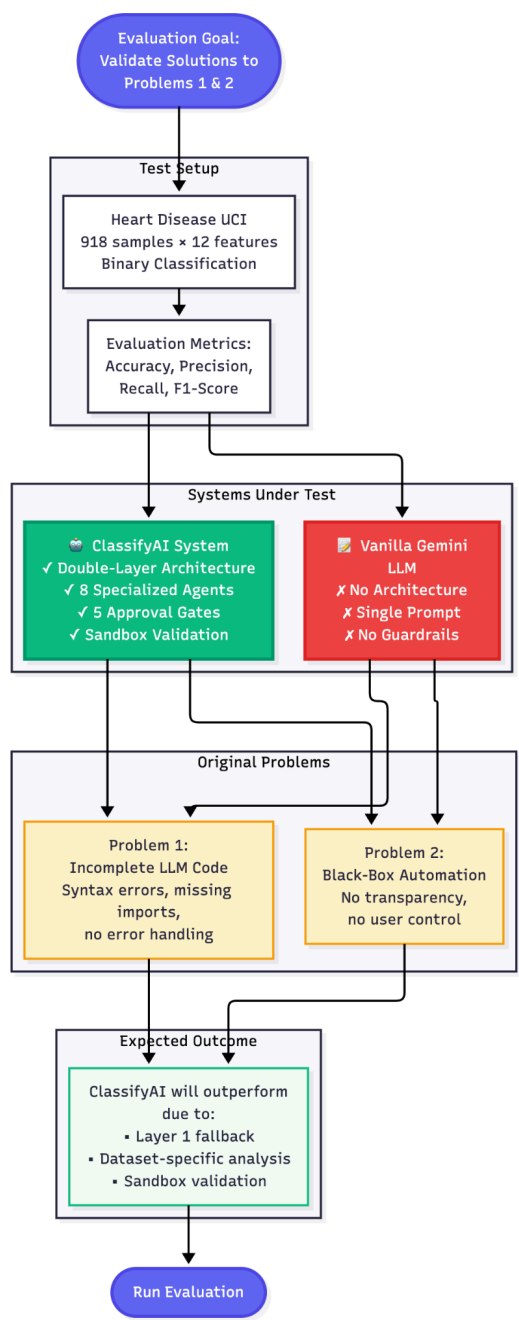


Figure A5: System Level Evaluation Strategy

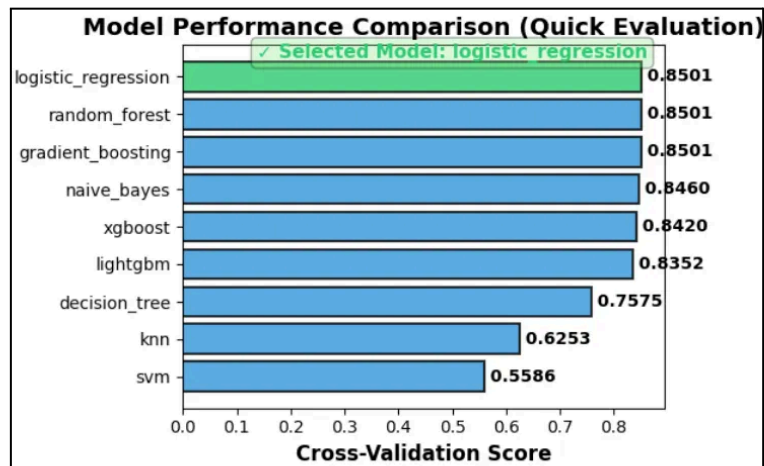


Figure A6: ML Builder Agent: Cross-validated results